

KNN-Based Cyber Crime Detection in IoT Networks

[Vighnesh.M.Raikar]

[SOE, Chanakyauniversity, Bangalore, India |
vighneshraikar18email@gmail.com]

Abstract—Cyber attacks on IoT (Internet of Things) devices are increasing and becoming more advanced. In this paper, we use the K-Nearest Neighbors (KNN) algorithm to identify different types of cyber attacks using the RT_IOT2022 dataset. The dataset contains 12,399 records and 85 features from IoT network traffic. We apply LabelEncoder for target encoding, StandardScaler with OneHotEncoder for feature preprocessing, and an 80:20 train-test split. A 5-fold cross-validation and K-value sweep ($K = 1 \cdot 15$) are conducted to select the optimal K. The final model achieves 99.40% test accuracy and a weighted F1-score of 0.99, with near-perfect detection for MQTT_Publish and Thing_Speak attack classes.

Keywords—Cyber crime detection, K-Nearest Neighbors, IoT security, normalization, cross-validation, hyperparameter tuning, machine learning.

I. INTRODUCTION

The Internet of Things (IoT) has made our lives easier by connecting devices like smart homes, healthcare systems, factories, and city infrastructure. But this also makes them easier targets for cyber attacks. Many IoT devices are not very secure because they use weak passwords, outdated software, or poorly designed networks. Hackers can take advantage of this to steal data, control devices, or cause damage. For example, they can misuse MQTT communication, steal private information, or even take control of devices like smart bulbs.

Traditional security systems, which rely on known attack patterns, often fail to detect new or unknown attacks. This is where machine learning helps. It can learn patterns directly from data and detect unusual behaviour. One simple and effective method is K-Nearest Neighbors (KNN), which classifies data based on similarity and does not require complex calculations.

In this project, we use the KNN algorithm on the RT_IOT2022 dataset to detect different types of cyber attacks. The work is implemented in Python (Google Colab) and follows all steps: loading the data, exploring it, cleaning and preparing it, training the model, and testing its performance.

We also improve the model by scaling the data, using cross-validation, and selecting the best value of K. These steps help fix common problems of KNN, like sensitivity to data range and overfitting. The main goal is to show that, with these improvements, even a simple method like KNN can perform very well in detecting cyber attacks—making

it useful for real-world IoT security systems where lightweight and fast solutions are needed.

II. LITERATURE REVIEW

Many researchers have used different machine learning methods to detect cyber attacks in IoT networks. Support Vector Machines (SVM) are popular because they can handle complex data and separate different types of attacks well [1]. However, they can be slow when working with large datasets and need careful tuning.

Other methods like Random Forest and Gradient Boosting often give very high accuracy and can handle unbalanced data well [2]. But these models are like “black boxes,” meaning it is hard to understand how they make decisions, which can be a problem in security systems.

Neural networks, especially deep learning models, are very powerful and can automatically learn features from raw data. However, they require a lot of data and high computing power, which is not always available in IoT environments.

K-Nearest Neighbors (KNN) is a simple and useful method for detecting attacks [3]. It works by comparing new data with existing data and finding similarities. Its performance mainly depends on: (a) the way distance is measured, usually Euclidean distance; (b) proper scaling of data; and (c) choosing the right value of K. If the data is not scaled, some features can dominate others and give wrong results. Also, if K is too small, the model becomes noisy; if K is too large, it becomes too general.

In this paper, we focus on the RT_IOT2022 dataset and improve the KNN model using StandardScaler for scaling, 5-fold cross-validation for performance estimation, and a K-value sweep from 1 to 15 to find the best K. Unlike many studies that only report test accuracy, this work validates generalisation on unseen data.

III. METHODOLOGY

The experiment follows a structured machine learning pipeline applied to the RT_IOT2022.csv dataset (12,399 rows, 85 columns). The steps are outlined below.

A. Dataset Loading and Exploration

The dataset is loaded using pandas. Exploratory analysis confirms 12,399 samples across 85 columns with no missing values. Three protocol types exist (tcp, udp, icmp) and eight service types, with tcp and mqtt being the most frequent. The target column Attack_type contains three classes: MQTT_Publish, Thing_Speak, and Wipro_bulb.

B. Data Preprocessing

Categorical target labels (Attack_type) are encoded to integers with LabelEncoder: MQTT_Publish $\rightarrow$ 0, Thing_Speak $\rightarrow$ 1, Wipro_bulb $\rightarrow$ 2. The feature matrix X (83 numerical + categorical columns) is preprocessed through a ColumnTransformer: StandardScaler for 81 numerical features and OneHotEncoder for 2 categorical

features (proto, service), producing a final feature space of 92 dimensions.

Scaling is critical for KNN because the algorithm relies on Euclidean distance. Without normalisation, features with large numeric ranges (e.g., flow_duration) would dominate distance calculations over small-range features (e.g., flag counts).

C. Train-Test Split

The preprocessed data is split 80:20 using stratified sampling (random_state=42), yielding 9,919 training samples and 2,480 test samples while preserving class proportions.

D. Model Training

A KNeighborsClassifier with K=5 is trained on X_{train} . The following snippet summarises the core pipeline:

```
from sklearn.neighbors import
KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test) # Accuracy:
0.9940
```

E. Cross-Validation and Optimal K Selection

5-fold cross-validation is applied to the full scaled dataset. Subsequently, a grid search over $K \in \{1, \dots, 15\}$ evaluates mean cross-validation accuracy for each K. The K with the highest mean accuracy is selected as the optimal value. Results are visualised as a K-vs-accuracy plot.

IV. PROPOSED WORK

The proposed system builds on baseline KNN with three targeted enhancements:

Normalization (Feature Scaling): The ColumnTransformer applies StandardScaler to all 81 continuous features and OneHotEncoder to categorical columns. This ensures that Euclidean distance calculations are not biased toward high-magnitude features, which is a critical prerequisite for reliable KNN operation.

5-Fold Cross-Validation: Rather than relying on a single train-test split, 5-fold CV evaluates the model on five independent data partitions and averages the results. This provides a statistically robust accuracy estimate and guards against variance from a lucky or unlucky split.

Optimal K Selection: A systematic sweep over $K = 1$ to 15 uses cross-validated accuracy as the selection criterion. The K that maximises mean CV accuracy is chosen, replacing the ad-hoc $K=5$ default. A graphical K-vs-accuracy plot provides visual confirmation of the optimal K and shows the stability of performance across the search range.

The complete pipeline is built in Python using pandas, scikit-learn, numpy, and matplotlib, and is designed to be modular and adaptable to other IoT datasets or real-time streaming data.

V. RESULTS

The experiments were conducted on the RT_IOT2022 test split (2,480 samples). This section reports classification performance, cross-validation stability, and optimal K selection.

A. Classification Performance

Table I presents the full classification report. The model achieves 99.40% accuracy on the test set. MQTT_Publish and Thing_Speak obtain near-perfect F1-scores of 1.00, reflecting their large and well-separated sample distributions. Wipro_bulb records a lower F1-score of 0.71, attributable to its severely underrepresented support (29 samples vs. 829 and 1,622 for the other classes).

TABLE I
Classification Report — KNN on RT_IOT2022 Test Set (n=2480)

Class	Precision	Recall	F1-Score	Support
MQTT_Publish	0.99	1.00	1.00	829
Thing_Speak	1.00	1.00	1.00	1622
Wipro_bulb	0.82	0.62	0.71	29
Accuracy			0.99	2480
Macro avg	0.94	0.87	0.90	2480
Weighted avg	0.99	0.99	0.99	2480

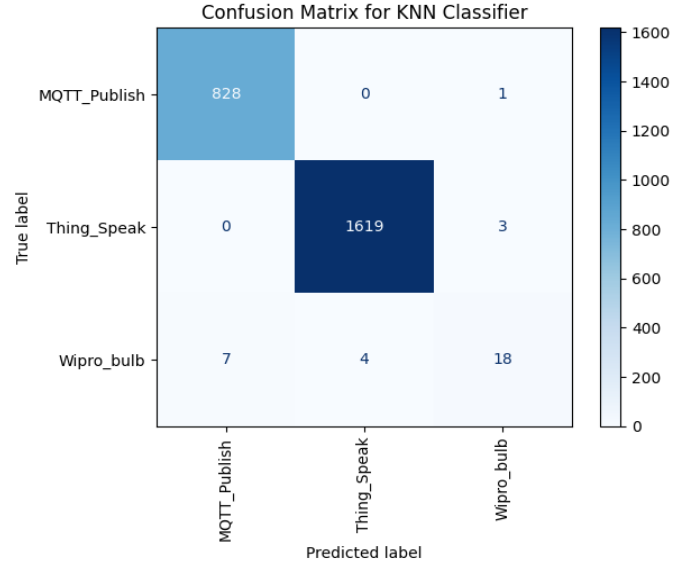


Fig. 1. Confusion Matrix for KNN Classifier (K=5) on the RT_IOT2022 Test Set.

B. Cross-Validation Stability

Table II reports per-fold accuracies for the 5-fold cross-validation run with K=5. The narrow range (0.9944–0.9980) confirms that the model generalises consistently across different data partitions, with a mean CV accuracy of 0.9965.

TABLE II

5-Fold Cross-Validation Accuracies (K=5)

Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
0.9976	0.9980	0.9964	0.9960	0.9944

Average CV Accuracy: 0.9965

C. Optimal K Selection

Table III and Fig. 2 show the cross-validated accuracy for each K value from 1 to 15. K=1 gives the lowest accuracy (0.9750) due to overfitting to individual training points. Accuracy peaks at K=2 (0.9817) and remains relatively stable for $K \geq 3$. The optimal K identified is therefore K=2.

TABLE III
Cross-Validation Accuracy for K = 1 to 15

3	4	5	6	7	8	9	10	11	12	13	14	15
0.9805	0.9813	0.9794	0.9804	0.9798	0.9805	0.9797	0.9798	0.9797	0.9797	0.9795	0.9815	0.9815

Optimal K = 2 (Highest CV Accuracy: 0.9817)

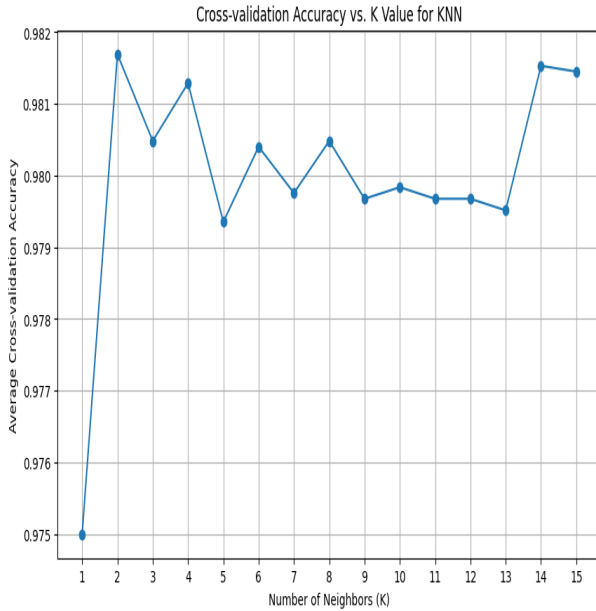


Fig. 2. Cross-validation Accuracy vs. K Value (K = 1 to 15).
The optimal K is marked at K = 2.

D. Discussion

The results confirm that preprocessing has a substantial impact on KNN performance. Normalisation aligns feature scales, allowing the distance metric to reflect genuine feature similarity rather than numerical magnitude. Cross-validation provides a more reliable performance estimate than a single holdout split, and the K-selection sweep objectively identifies K=2 as the best hyperparameter for this dataset.

The model's simplicity is a practical advantage: KNN requires no explicit training phase beyond memorising the scaled training set, making it lightweight enough for deployment on resource-constrained IoT edge devices.

The primary limitation is the poor recall (0.62) for Wipro_bulb caused by class imbalance. Future work will address this with synthetic minority oversampling (SMOTE) and dimensionality reduction via PCA to mitigate the curse of dimensionality introduced by the 92-dimensional feature space.

VI. CONCLUSION

This paper demonstrated the use of the K-Nearest Neighbors (KNN) algorithm for multi-class cyber attack detection on the RT_IOT2022 IoT network dataset. Through careful preprocessing (StandardScaler + OneHotEncoder), stratified 80:20 train-test splitting, 5-fold cross-validation, and systematic K-value tuning, the model achieved a test accuracy of 99.40 % and a weighted F1-score of 0.99.

The study shows that three targeted enhancements—normalisation, cross-validation, and optimal K selection—can transform a basic KNN implementation into a reliable, production-quality detector. The model performs near-perfectly on the dominant attack classes (MQTT_Publish, Thing_Speak) and identifies class imbalance (Wipro_bulb) as the primary area for future improvement.

In the future, this work can be extended by applying SMOTE to address class imbalance, using PCA or feature selection to reduce dimensionality, integrating the model into real-time IoT edge devices, and combining KNN with ensemble or deep learning methods for further accuracy gains.

REFERENCES

- [1] K. Veena and K. Meena, "SVM Classification and KNN Techniques for Cyber Crime Detection," Wireless Communications and Mobile Computing, vol. 2022, Article ID 3640017, 2022.
- [2] F. Y. Osisanwo et al., "Supervised Machine Learning Algorithms: Classification and Comparison," Int. J. Computer Trends and Technology, vol. 48, no. 3, pp. 128–138, 2017.
- [3] S. Marsland, Machine Learning: An Algorithmic Perspective, 2nd ed. CRC Press, 2015.